
Lab W1: Environment Setup – QEMU and Raspberry Pi

Introductory lab, not tied to a chapter

A note on how this assignment was developed

This assignment was drafted with AI help and has not been fully tested by the instructor. It should be accurate, but something might still be wrong. If you find an error, please tell your instructor; that helps later versions, though finding errors is not the point of this assignment.

Purpose

Every assembly lab this term asks you to build and run code for two different instruction set architectures: x86-64 and ARM. This assignment has no architecture content of its own. Its only job is to get your group's tools working so that nothing later stalls on a broken setup. Work through it as a group of four, but confirm that all four of you can independently reproduce the results on your own machine before you submit.

Estimated time

About one to two hours per person, spread across the week. Some of that time is waiting on downloads and installs, not active work.

1 Part A – QEMU on your own laptop

Each of you should install QEMU user-mode and system emulation, along with a cross-compiler toolchain, so you can assemble and run both x86-64 and ARM (AArch64) binaries without owning matching hardware.

1. Install QEMU (`qemu-user`, `qemu-system-arm`, and `qemu-system-x86` packages, or your distribution's equivalents).
2. Install a cross-compilation toolchain capable of producing AArch64 binaries (for example `gcc-aarch64-linux-gnu` and `binutils-aarch64-linux-gnu` on Debian-based systems).
3. Write a minimal “hello world” program in C.
4. Compile it natively for your laptop's own architecture and confirm it runs directly.
5. Cross-compile the same program for AArch64 and run the resulting binary under `qemu-aarch64` (user-mode emulation).

Table 1: Part A checklist

Checklist item	Confirmed by all 4?
QEMU installed and <code>qemu-system-x86_64 -version</code> runs	
AArch64 cross-compiler installed	
Native hello-world compiles and runs	
Cross-compiled AArch64 hello-world runs under QEMU	

2 Part B – Raspberry Pi units

Pair up (two pairs of two) and get one Raspberry Pi unit running per pair.

1. Boot the Pi and connect to it (directly or over SSH).
2. Run `uname -m` and record the output.
3. Run `cat /proc/cpuinfo` and note the `model name` or `Hardware` field near the top.
4. Compile and run the same hello-world program directly on the Pi (native compilation, no cross-compiler needed here).

3 Part C – Short reflection

Answer these as a group in a few sentences each. These are meant to get you thinking, not to be graded on length.

1. What did `uname -m` report on the Raspberry Pi, and what does that string actually name?
2. You cross-compiled for AArch64 on your laptop in Part A but compiled natively on the Pi in Part B. In your own words, why does a cross-compiler exist at all? What problem does it solve that a regular compiler cannot?
3. QEMU let you run an AArch64 binary on an x86-64 laptop. Based on what you observed, what do you think QEMU is actually doing to make that possible? A guess grounded in what you saw is fine here; we will cover this properly once we reach later chapters.

What to submit

Submit your completed Part A checklist, the `uname -m` and `cpuinfo` output from Part B, and direct answers to the Part C questions. No formal write-up is needed, just clear answers. Include at least one screenshot showing the QEMU setup working and one showing the Raspberry Pi setup working. You do not need a screenshot from every group member; if you worked through setup together, one screenshot of each is enough. If any step did not work for someone in the group, say so and describe what you tried. That is useful information, not a problem.

Resources

- QEMU documentation home – <https://www.qemu.org/documentation/>
- QEMU User Mode Emulation guide – <https://www.qemu.org/docs/master/user/index.html>

-
- Raspberry Pi official documentation – <https://www.raspberrypi.com/documentation/>
 - Arm Learning Paths, installing a GCC cross-compiler for Arm Linux targets – <https://learn.arm.com/install-guides/gcc/cross/>
 - Debian Wiki, CrossCompiling – <https://wiki.debian.org/CrossCompiling>
-

A note on these resources

These links were found with help from an AI system and web search, not from the instructor's own prior use. They look relevant, but none have been personally verified by the instructor. Treat them as a starting point, and let your instructor know if one is broken, off-topic, or wrong.
